

Deep Learning for Data Science

Djallel Bouneffouf

Professor, Columbia University

Research Staff Member (RSM), IBM Research

New York, USA

Lecture Overview

- **Who am I? Who are you?**
- **Course introduction and expectations**
- **Final project**
- **Introduction to deep learning**
 - Perceptron
 - Logistic Regression
 - Binary Cross-Entropy (BCE)
- **Homework**

Who am I?

db3914@columbia.edu

On Slack (←more responsive here)

Nazgul Maksutkhan,

Nami Jain

Sreya Karuturi

•
•
•

Course Introduction and Expectations

- Nature of the course
 - A blend of theory and practice
 - The focus is on building a strong, thorough understanding of the fundamentals, even if that means covering advanced material more briefly or at a higher level
 - learn by doing, reflecting, and iterating Final Project
 - IBM problem setting to give you exposure to industry real life problems
- Expectations
 - Interact. Ask questions in or outside of class if you need help.
 - Focus (we'll take plenty of breaks)
 - Learn Python if you have to
 - Complete homework within the week (seriously - you get a zero otherwise)
 - Be prepared to present your work
- No laptops or phones out! They're too distracting to you and to me.
 - unless you have a documented disability and have received departmental permission

More expectations

- Join the slack (you should have an invitation)
- Homework
 - ~3 hours a week
 - reading
 - Questions to confirm and deepen understanding
 - Practical work (coding)

Submitting Homework

- Submit your homework to
- Create one top-level document to submit a link to. Typically this will be a Google Doc that you share with **Edit** access to
- **Nazgul Maksutkhan,**
nm3663@columbia.edu
- **Nami Jain**
njain@barnard.edu
- **Sreya Karuturi**
sk5884@columbia.edu
- we need edit access to verify you've not changed after the deadline. If you don't share with edit access you won't receive a grade.
- Don't use LLMs like ChatGPT. Don't be tempted. You won't learn the content will likely fail.

Course Introduction

- Course introduction; perceptron; linear classifiers; logistic regression; sigmoid; cross-entropy; decision boundaries; first implementation.

Perceptron

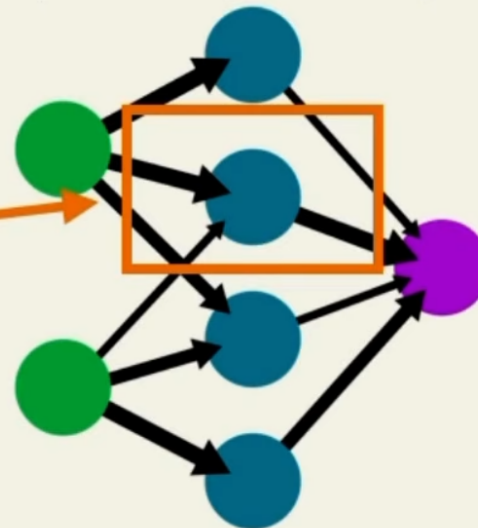
A simple neural network

input
layer

hidden
layer

output
layer

The Perceptron



Perceptron

Frank Rosenblatt



Image: Cornell Chronicle

Mark I Perceptron



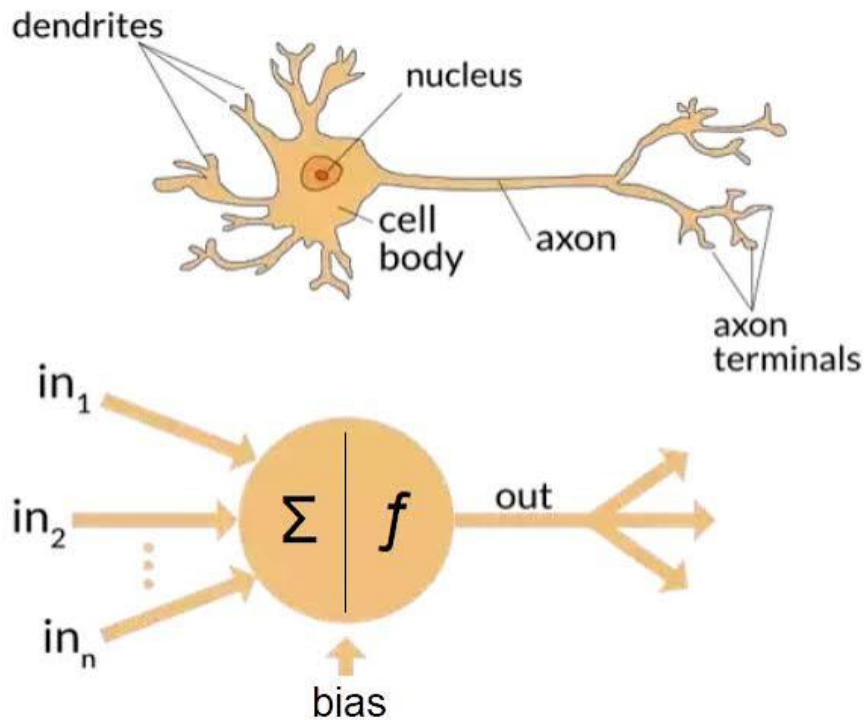
Image: Wikipedia

IBM's hardware—specifically the IBM 704 computer—was used by Frank Rosenblatt to simulate the very first perceptron program in 1957. [Wikipedia](#) +1



The practical implementation, was built to process visual input from a grid of photocells and classify simple shapes or characters into distinct categories.

Perceptron



At the top, we see a biological neuron. A neuron has three important components for our discussion:

- **Dendrites** receive signals from other neurons.
- The **cell body** combines these signals.
- The **axon** sends the resulting signal to other neurons.

“The artificial neuron is inspired by this basic idea: receive several inputs, combine them, make a decision, and produce an output.”

Perceptron

1. What is a Perceptron?

A perceptron is a **binary linear classifier**.

It takes several input features:

$$x_1, x_2, \dots, x_n$$

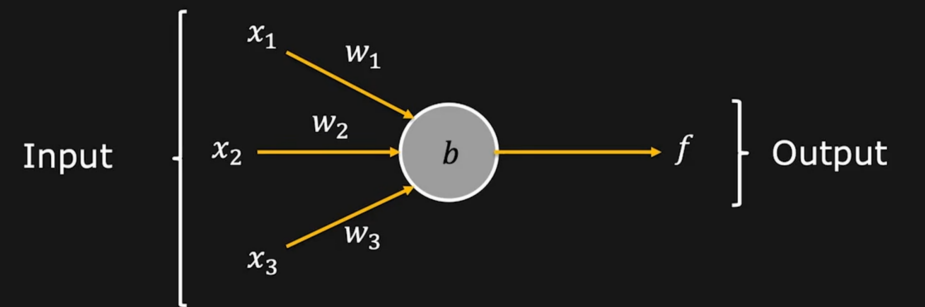
multiplies each by a weight:

$$w_1, w_2, \dots, w_n$$

adds them together, and makes a decision.

The basic equation is:

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$



$$f = \begin{cases} 0 & \text{if } \sum_j w_j x_j \leq -b \\ 1 & \text{if } \sum_j w_j x_j > -b \end{cases} \quad f = \begin{cases} 0 & \text{if } w \cdot x + b \leq 0 \\ 1 & \text{if } w \cdot x + b > 0 \end{cases}$$

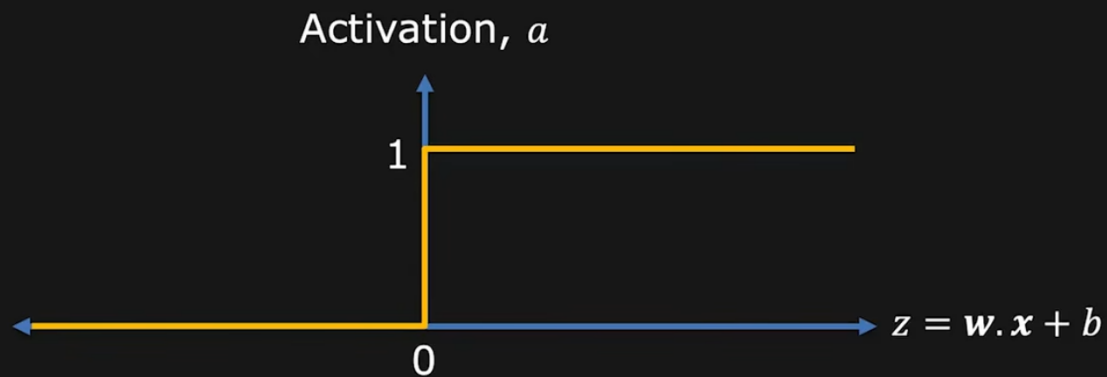
where, $w_j = \text{weights}$

$b = \text{bias (threshold)}$

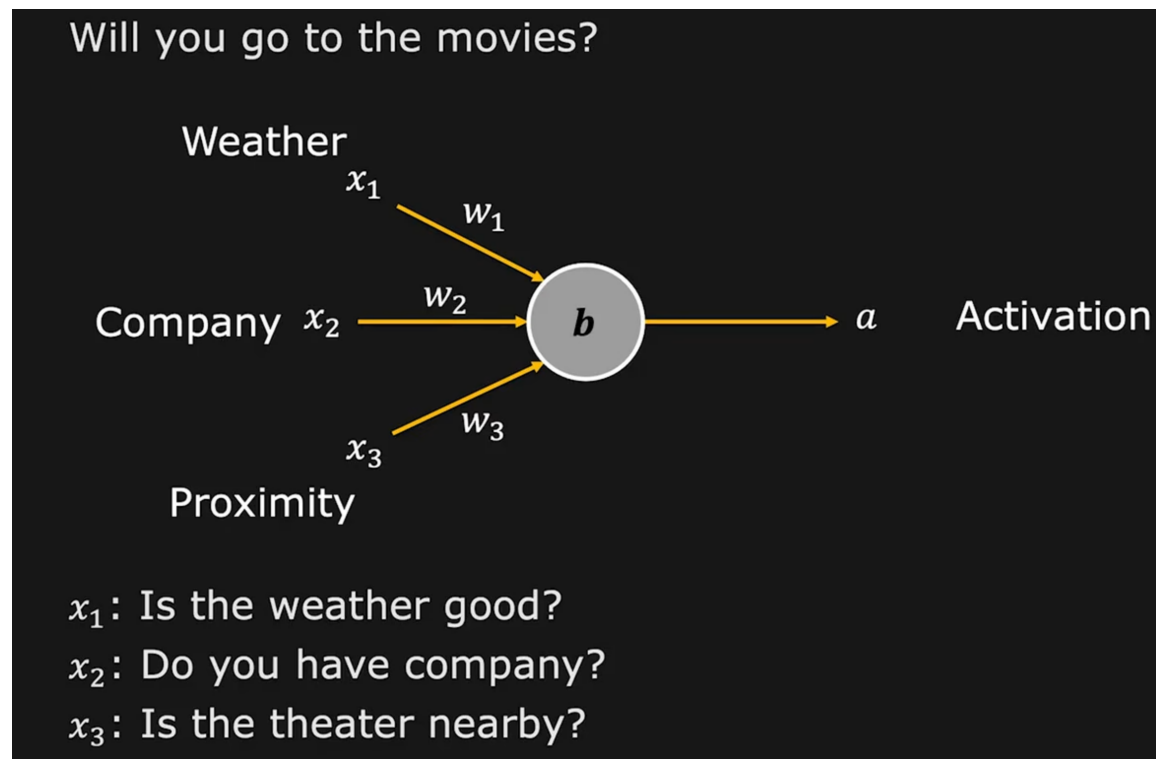
Activation Function of the Perceptron

Let: $z = w \cdot x + b$

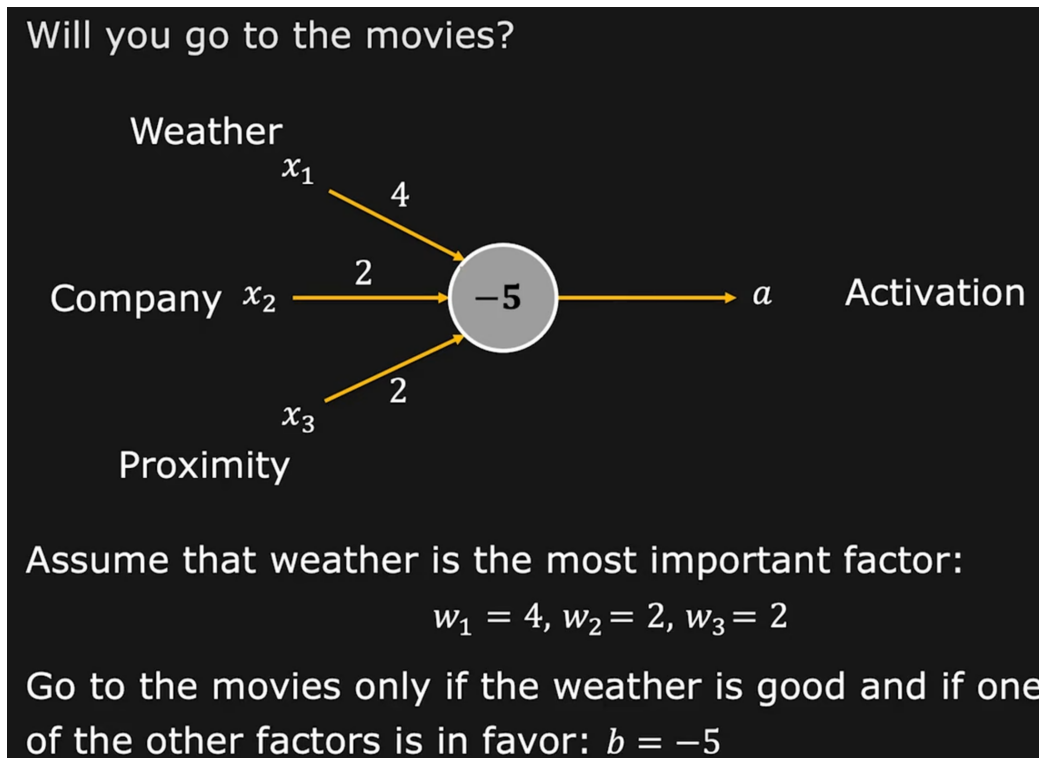
$$\text{Activation: } a = f(z) = \begin{cases} 0 & \text{if } z \leq 0 \\ 1 & \text{if } z > 0 \end{cases}$$



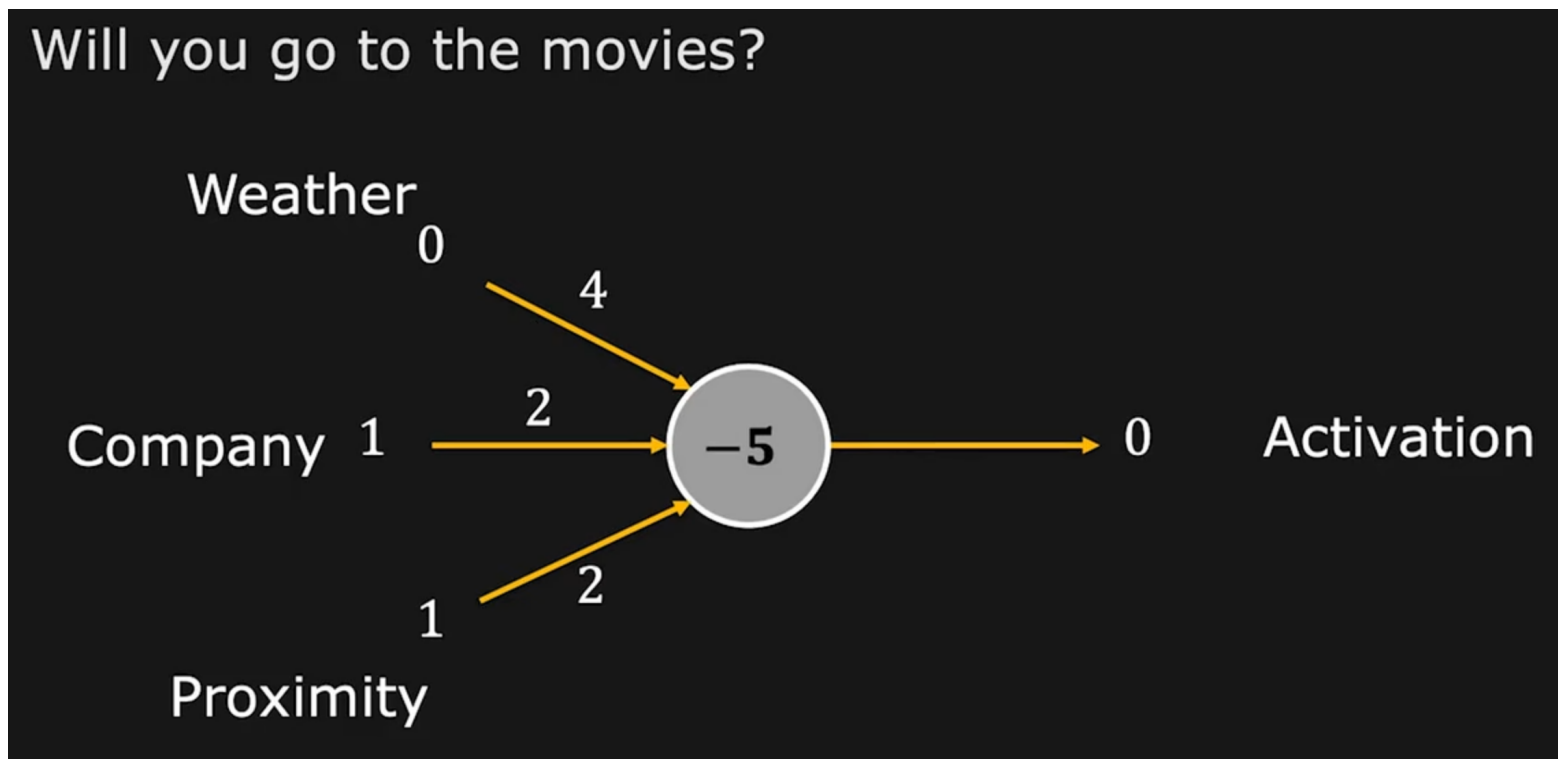
Perceptron: Example



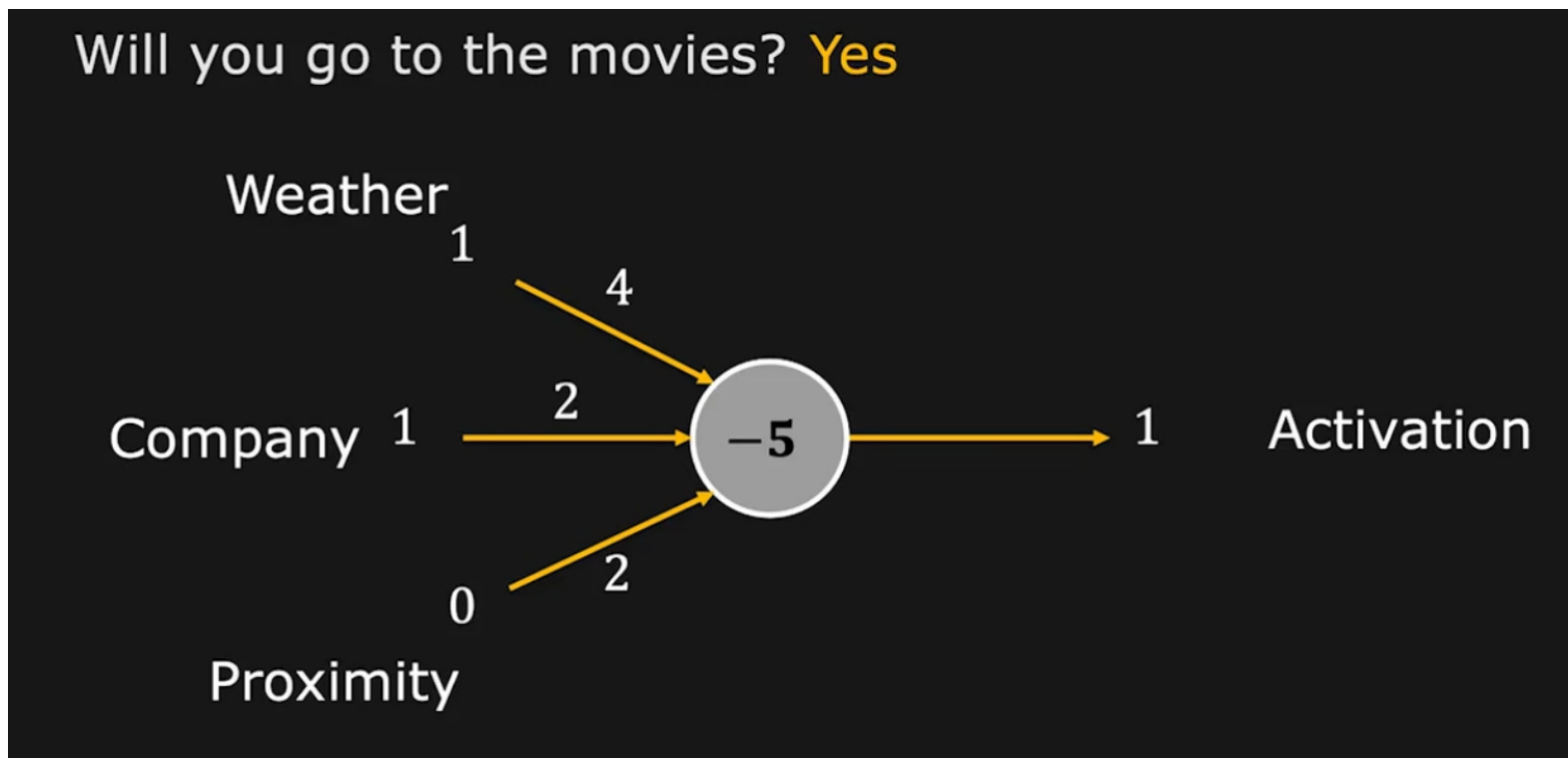
Perceptron: Example



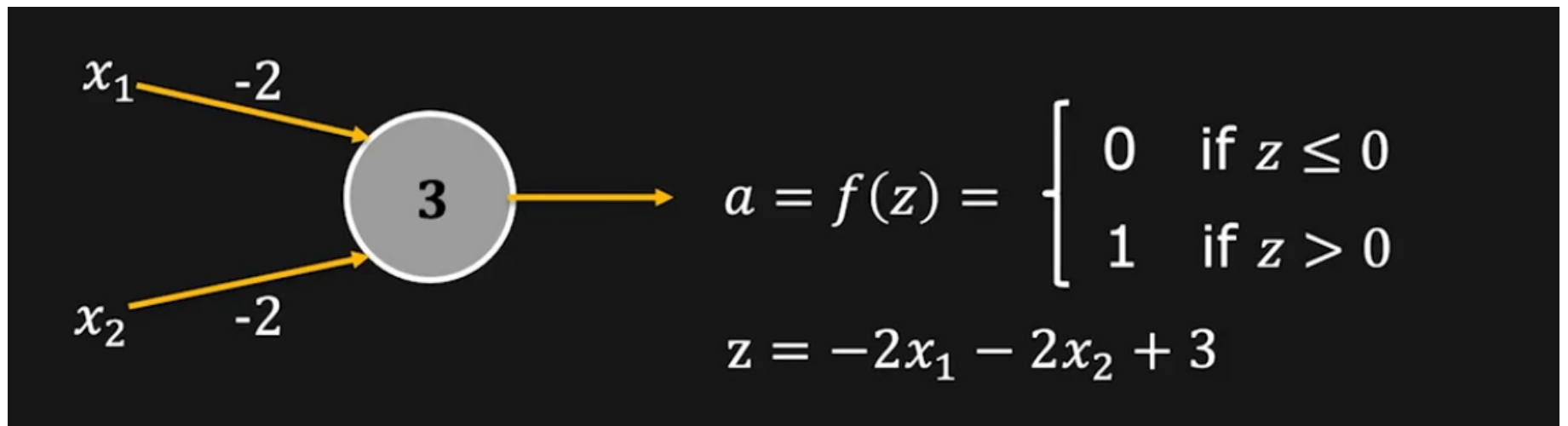
Perceptron: Example



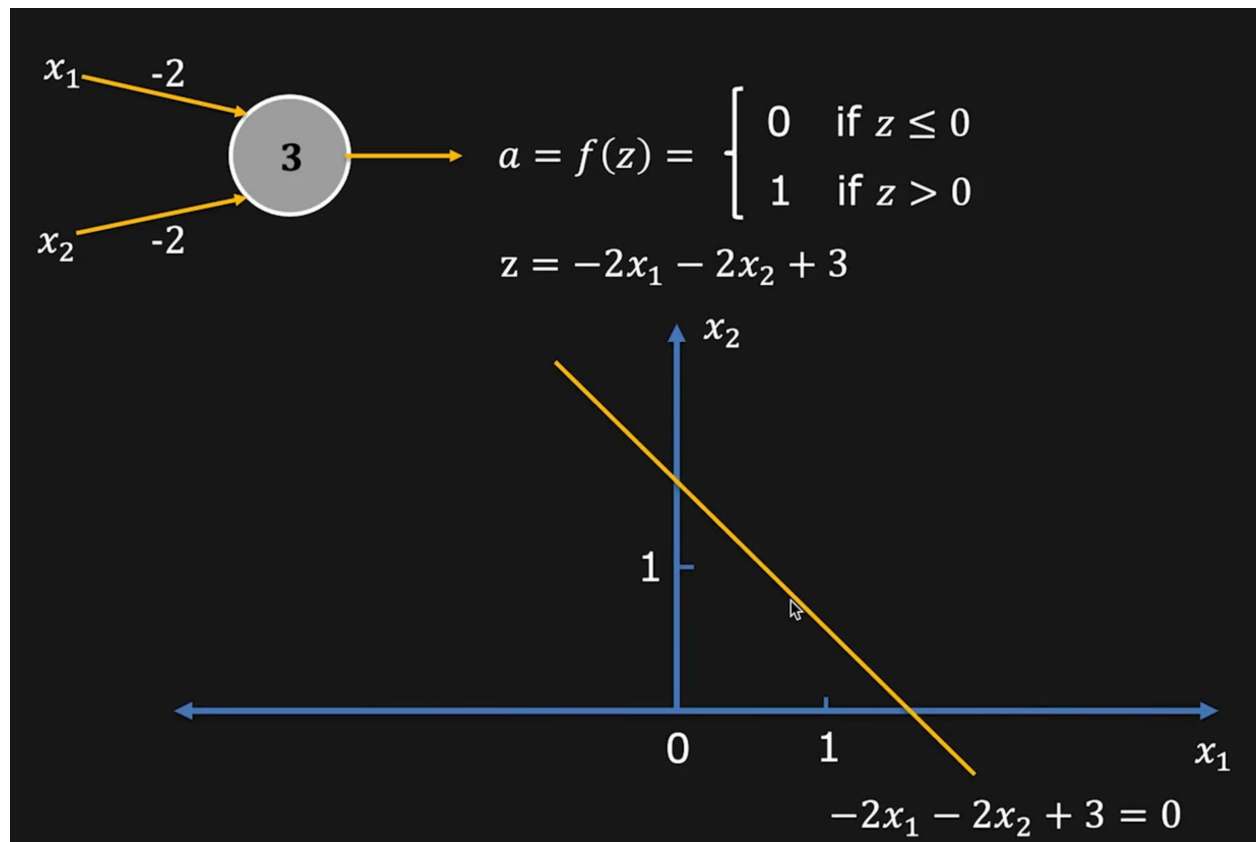
Perceptron: Example



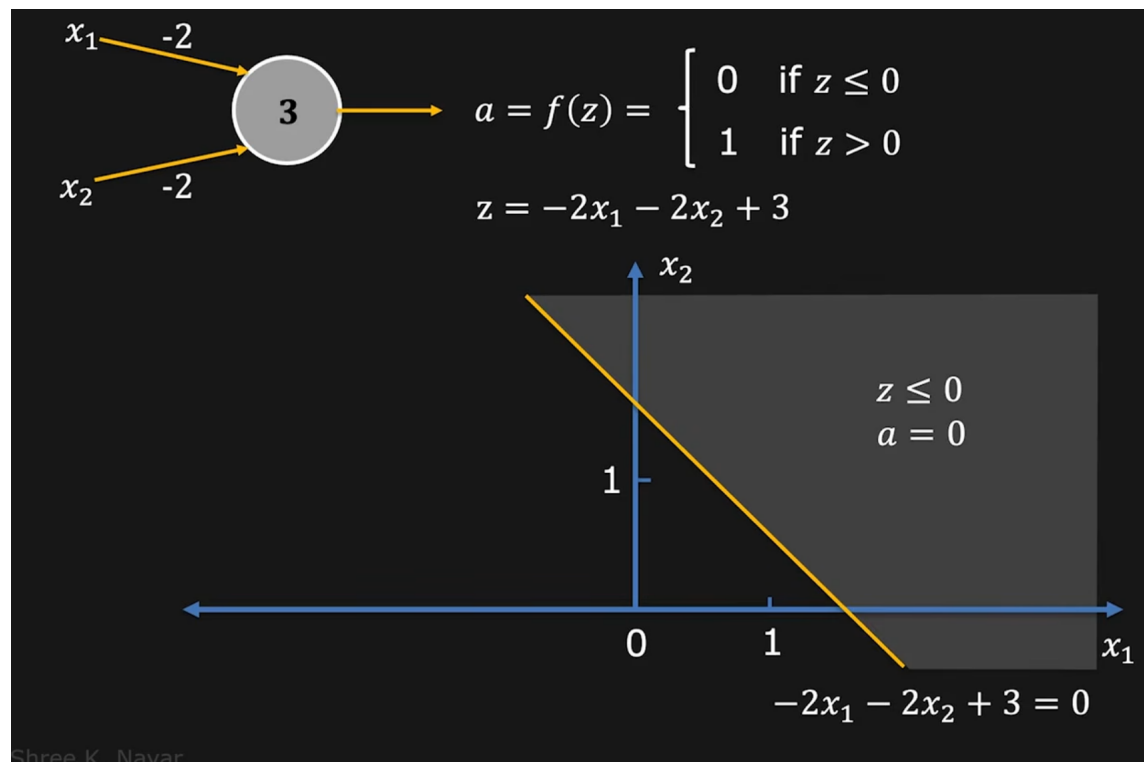
Perceptron as linear classifier



Perceptron as linear classifier



Perceptron as linear classifier



Perceptron Algorithm

The perceptron learns through a simple loop:

1. Make a prediction

$$\hat{y} = \text{step}(w^T x + b)$$

↓

2. Compare prediction with reality

$$\text{error} = y - \hat{y}$$

↓

3. If wrong, change the weights

$$w_i \leftarrow w_i + \eta(y - \hat{y})x_i$$

↓

4. Repeat for all training examples

Example: Will you go to the movies?

We have 3 features:

- x_1 = Weather is good? $\rightarrow 1$ = Yes, 0 = No
- x_2 = Friends are going? $\rightarrow 1$ = Yes, 0 = No
- x_3 = Movie theater is close? $\rightarrow 1$ = Yes, 0 = No

The target y :

- $y = 1$: Go to the movies
- $y = 0$: Don't go

Suppose we start with:

$$w_1 = 0, \quad w_2 = 0, \quad w_3 = 0, \quad b = 0$$

The perceptron calculates:

$$z = w_1x_1 + w_2x_2 + w_3x_3 + b$$

and predicts:

$$\hat{y} = \begin{cases} 1 & z > 0 \\ 0 & z \leq 0 \end{cases}$$

Training example 1

Suppose:

Weather	Friends	Close	Actual y
Good	Yes	Yes	1

So:

$$x = (1, 1, 1), \quad y = 1$$

With our initial weights:

$$z = 0(1) + 0(1) + 0(1) + 0 = 0$$

Therefore:

$$\hat{y} = 0$$

Wrong! The person actually went to the movie.

Training

The perceptron updates its weights:

$$w_i \leftarrow w_i + \eta(y - \hat{y})x_i$$

Let's use learning rate $\eta = 1$.

Since:

$$y - \hat{y} = 1 - 0 = 1$$

we get:

$$w_1 = 0 + 1(1)(1) = 1$$

$$w_2 = 0 + 1(1)(1) = 1$$

$$w_3 = 0 + 1(1)(1) = 1$$

and:

$$b = 0 + 1(1) = 1$$

So now:

$$w = (1, 1, 1), \quad b = 1$$

Training example 2

Weather	Friends	Close	Actual
Bad	Yes	No	0

So:

$$x = (0, 1, 0), \quad y = 0$$

Calculate:

$$z = (1)(0) + (1)(1) + (1)(0) + 1 = 2$$

Therefore:

$$\hat{y} = 1$$

Wrong again! The person did not go.

Update:

$$y - \hat{y} = 0 - 1 = -1$$

Therefore:

$$w_1 = 1 + (-1)(0) = 1$$

$$w_2 = 1 + (-1)(1) = 0$$

$$w_3 = 1 + (-1)(0) = 1$$

$$b = 1 + (-1) = 0$$

Now:

$$w = (1, 0, 1), \quad b = 0$$

Training example 3

Weather	Friends	Close	Actual
Good	No	Yes	1
$x = (1, 0, 1), \quad y = 1$			
Calculate:			
$z = (1)(1) + (0)(0) + (1)(1) + 0 = 2$			
Therefore:			
$\hat{y} = 1$			
Correct!			
No update is necessary.			

From Perceptron to Logistic Regression

A Problem with the Perceptron

Perceptron gives:

- 0 or 1

But what if we want:

- Probability?
- Confidence?
- Smooth learning?

Probability Is Useful

Instead of:

Positive

We could say:

Probability of positive = 0.92

This is much more informative.

$$x \rightarrow w^T x + b \rightarrow \sigma(z) \rightarrow P(y = 1|x)$$

Logistic Regression

- A linear model for classification
- Computes a linear score
- Converts the score into a probability

Linear Score

First calculate:

$$z = w^T x + b$$

Example:

$$z = 2.5$$

But z is not a probability.

We Need a Function

We need a function that converts:

$$(-\infty, +\infty)$$

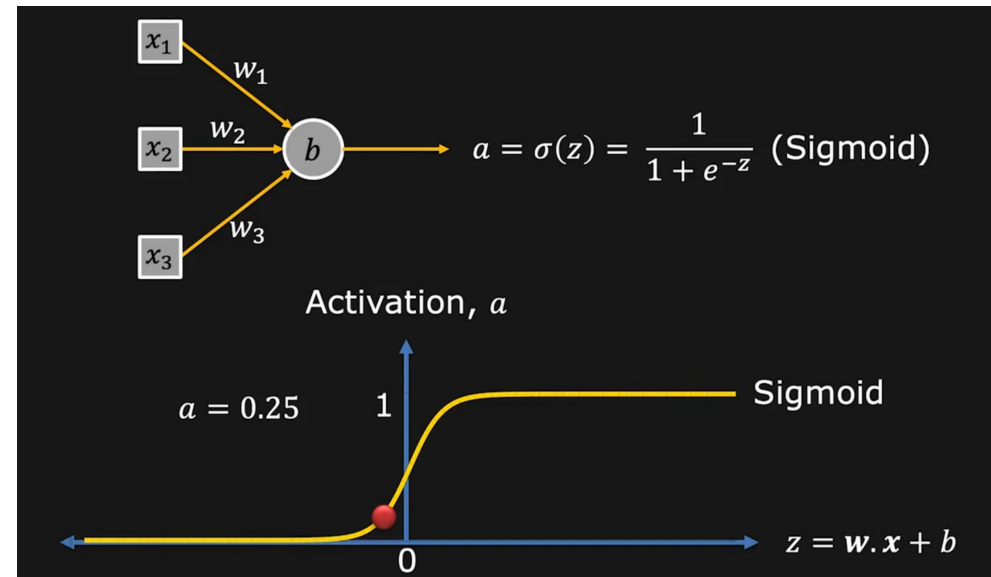
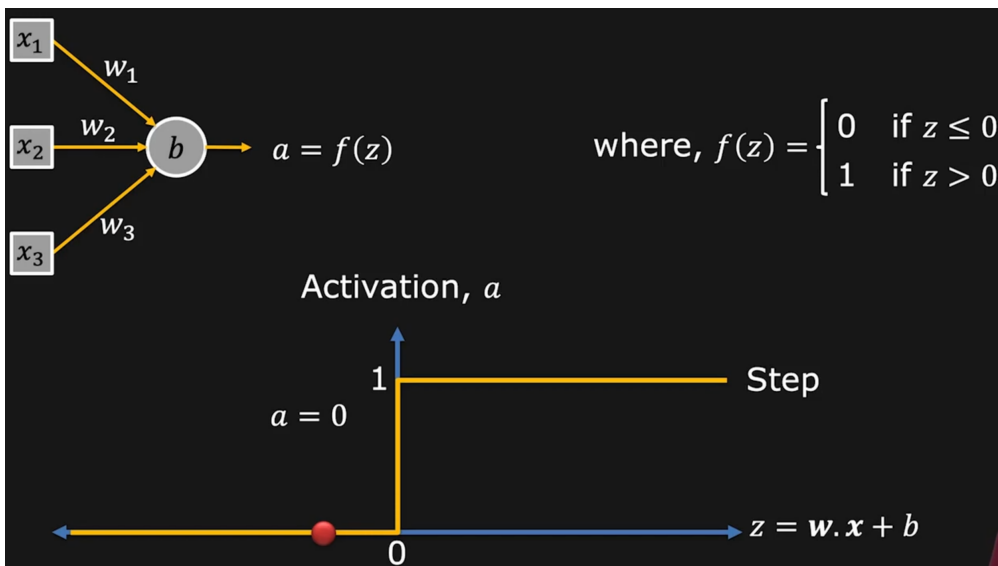
into:

$$(0, 1)$$

The solution:

Sigmoid function

Sigmoid Function



Sigmoid Function

	Perceptron	Logistic Regression
Score	$w^T x + b$	$w^T x + b$
Activation	Step	Sigmoid
Output	0/1	Probability
Loss	Perceptron update	Cross-entropy

Binary Cross-Entropy (BCE)

Binary Cross-Entropy (BCE)

Binary Cross-Entropy is the loss function commonly used when a neural network performs **binary classification** — when the answer is either **0 or 1**.

For your **image classification** lecture, think:

“Is this image a cat or not a cat?”

- $y = 1 \rightarrow \text{Cat}$
- $y = 0 \rightarrow \text{Not cat}$

The model doesn't directly output 0 or 1. It outputs a **probability**:

$$\hat{y} = P(y=1|x)$$

Example

Image	True label	Model prediction
 Cat	1	0.90
 Cat	1	0.60
 Dog	0	0.10
 Dog	0	0.80

For example:

The question BCE asks is:

How wrong was the model's probability prediction?

The equation

$$L = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

Think of the equation as having **two doors**:

$$\underbrace{y \log(\hat{y})}_{\text{positive class}} + \underbrace{(1 - y) \log(1 - \hat{y})}_{\text{negative class}}$$

The true label decides which door is open.

If $y = 1$:

$$L = -\log(\hat{y})$$

Only the positive-class prediction matters.

If $y = 0$:

$$L = -\log(1 - \hat{y})$$

Only the negative-class prediction matters.

Now compare predictions:

If the true answer is 1

Suppose the image really **is a cat**:

$y = 1$

Then:

$$L = - \left[1\log(\hat{y}) + 0\log(1 - \hat{y}) \right]$$

so:

$$L = -\log(\hat{y})$$

Now compare predictions:

True label	Prediction	BCE loss
1	0.99	0.01
1	0.90	0.11
1	0.70	0.36
1	0.50	0.69
1	0.10	2.30
1	0.01	4.61

Important idea:

If the model is confidently correct → small loss.

If the model is confidently wrong → huge loss.

If the true answer is 0

- Suppose the image is a **dog**:
- $y = 0$
- Then:

$$L = - \left[0 \log(\hat{y}) + 1 \log(1 - \hat{y}) \right]$$

- Therefore:
- $L = -\log(1 - \hat{y})$

For example:

True label	Prediction "cat"	BCE loss
0	0.01	0.01
0	0.10	0.11
0	0.30	0.36
0	0.50	0.69
0	0.90	2.30
0	0.99	4.61

Again, confidently wrong = very large penalty.

Why do we use the logarithm?

- Imagine two models predict that a dog is a cat:
- **Model A:**
- $\hat{y} = 0.6$
- **Model B:**
- $\hat{y} = 0.99$
- Both are wrong, but Model B is **much more confidently wrong**.
- BCE strongly penalizes Model B:
- $L_A = -\log(1 - 0.6) = 0.916$
- $L_B = -\log(1 - 0.99) = 4.605$
- So BCE communicates:
- **"You can be wrong, but don't be extremely confident when you're wrong."**
- That's particularly useful when training probabilistic classifiers.

How it trains the neural network

During training, the network tries to **minimize the BCE loss**.

For every training example:

prediction → BCE loss → gradient → update weights

Eventually, the network learns weights that produce probabilities close to the correct labels.

For a dataset of N examples:

$$J = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

This is simply the **average BCE over the training examples**.

- **Binary Cross-Entropy** measures how well predicted probabilities match binary labels, heavily penalizing confident incorrect predictions.

Perceptron, Logistic Regression & BCE

- **Dataset:** Forest Cover Type dataset from the **UCI Machine Learning Repository** Approximately **581,000 observations** with multiple environmental features.
- **Objective:** Use the dataset to build and compare two **binary classification models**.
- **1. Data Preparation**
 - Load the Forest Cover Type dataset.
 - Choose **two cover types** and convert the problem into **binary classification**.
 - Split the data into **training (80%)** and **test (20%)** sets.
 - Standardize the features.
- **2. Perceptron**
 - Train a **Perceptron** classifier.
 - Report **accuracy, precision, recall, and F1-score**.
 - Visualize the decision boundary using two selected features.
- **3. Logistic Regression**
 - Train a **Logistic Regression** classifier on the same data.
 - Report **accuracy, precision, recall, and F1-score**.
 - Visualize the decision boundary.
- **4. Binary Cross-Entropy (BCE)**
 - Calculate the **BCE loss** on the test set.
 - Plot the BCE loss as a function of the predicted probability.
- **5. Comparison**
 - Create a table comparing.

Final Project Deliverables

- **1. Research Paper —December 01**
- **ACM Web Conference (WWW) Format**
- **6-8 pages** following the ACM Web Conference paper format
- Reproduce a selected **Deep Learning research paper**
- Identify a **limitation or research gap**
- Propose an **algorithmic extension** using Deep Learning and/or LLMs
- Experimental evaluation against strong baselines
- Ablation studies and analysis
- Discuss limitations and future work
- **Code must be reproducible**
- **3. Final Presentation — December 14**
- Problem & motivation
- Original paper → limitation → proposed solution
- Technical contribution
- Experimental results
- Live/demo or code walkthrough
- Questions & discussion
- **Goal:** Produce a **research-quality paper + reproducible GitHub project** that could serve as the basis for a submission to **The ACM Web Conference (WWW)**.

2. GitHub Repository

Complete source code
README with setup and instructions
Dataset / data preparation
Baseline implementation
Proposed method
Experiment scripts and configurations
Results, plots, and tables
requirements.txt or environment specification

Final Project : Improve on Top of Existing Work

Starting Point

“A Neural Networks Committee for the Contextual Bandit Problem”

Robin Allesiardo · Raphaël Féraud · Djallel Bouneffouf

Your Challenge

Reproduce the original method — then go beyond it.



Step 1 — Reproduce

Implement the Neural Networks Committee and reproduce the main experimental results.



Step 2 — Find a Gap

Identify a limitation, assumption, or unexplored setting.



Step 3 — Innovate

Propose a meaningful improvement:

- Better exploration strategy
- More efficient neural architecture
- New committee/ensemble mechanism
- Improved uncertainty estimation
- New contextual-bandit algorithm
- Application to a new real-world problem



Step 4 — Evaluate

Compare **Original vs. Your Method** using rigorous experiments.



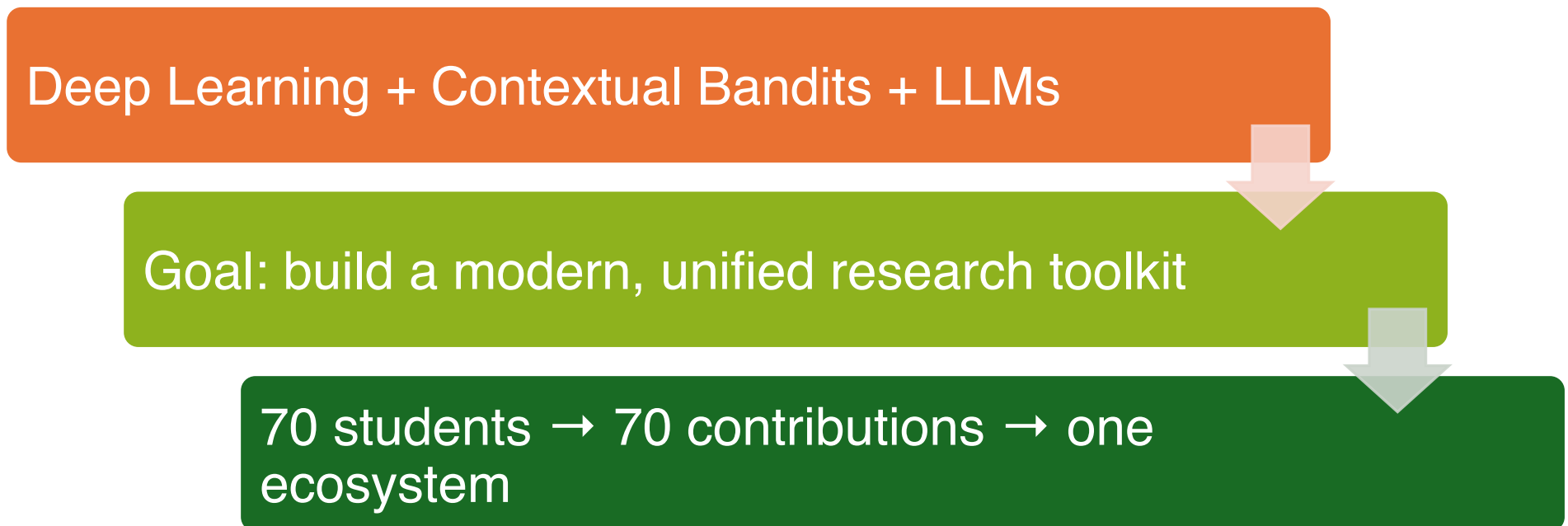
Research Question

Can you design a method that performs better, learns faster, or is more robust than the original approach?

Paper → Reproduce → Gap → Innovation → Experiment → Evidence

Final Project: From NeuralBandit to NeuralBandit++

Deep Learning + Contextual Bandits + LLMs



```
graph TD; A[Deep Learning + Contextual Bandits + LLMs] --> B[Goal: build a modern, unified research toolkit]; B --> C[70 students → 70 contributions → one ecosystem];
```

Goal: build a modern, unified research toolkit

70 students → 70 contributions → one ecosystem

Why are we starting from a 2014 paper?

- The paper introduced NeuralBandit for contextual bandits
- It uses neural networks to predict reward from context
- It addresses exploration vs. exploitation
- It explicitly considers non-stationary environments
- Your job is NOT simply to reproduce it — your job is to improve it

The original problem

- At time t , observe context x_t
- Choose one action k from K possible actions
- Only the reward of the chosen action is observed
- Learn online while making decisions
- Objective: maximize cumulative reward / minimize regret

A simple example: recommendation

- Context: user profile, recent clicks, time, device
- Actions: recommend Movie A, B, C, D
- Reward: click, watch, purchase, or satisfaction
- The algorithm must learn while it is serving recommendations

The key challenge: exploration vs. exploitation

- Exploit: choose the action currently predicted to be best
- Explore: try actions whose value is uncertain
- Too much exploitation → may miss better actions
- Too much exploration → wastes reward
- A good bandit learns while making decisions

What NeuralBandit does

- One neural network models each action
- Input: context
- Output: predicted probability/reward
- The selected action produces feedback
- The network is updated online
- The original paper also uses EXP3 for model selection

Why improve the original paper?

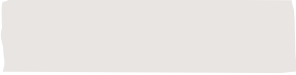
- 2014 neural networks were much smaller and simpler
- Modern deep learning provides transformers, attention, AdamW, MoE, self-supervised learning, etc.
- LLMs provide powerful contextual representations
- Modern applications include RAG, agents, recommendation and tool selection
- This gives us many research directions

Our research question

- How can modern deep learning and LLMs transform NeuralBandit into NeuralBandit++?
- We will study:
 - better representations
 - better exploration
 - better adaptation to changing environments
 - LLM-based decision making
 - agentic AI applications

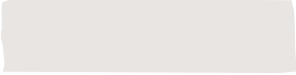
The project philosophy

- Paper → Reproduce → Compare → Identify limitation → Improve → Evaluate → Integrate
- Every project must start from a real baseline
- Every improvement must be experimentally justified



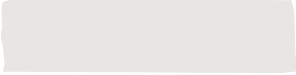
Track 1 — Deep Neural Architectures

- Deep MLP NeuralBandit
- Residual NeuralBandit
- CNN NeuralBandit
- LSTM NeuralBandit
- GRU NeuralBandit
- Transformer NeuralBandit
- Mixture-of-Experts NeuralBandit



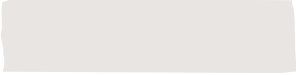
Track 2 — Modern Deep Learning Optimization

- Adam / AdamW
- Learning-rate schedules
- BatchNorm / LayerNorm
- Dropout
- Weight decay
- Deep ensembles
- Question: does modern optimization reduce regret or improve adaptation?



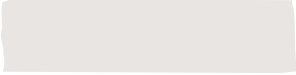
Track 3 — Representation Learning

- Autoencoder representations
- Variational autoencoders
- Contrastive learning
- Self-supervised learning
- Attention-based feature selection
- Learned latent context representations
- Question: can better representations make the bandit more sample-efficient?



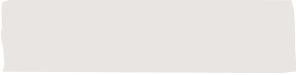
Track 4 — Deep Exploration

- Neural UCB
- Neural Thompson Sampling
- Deep ϵ -greedy
- Bayesian neural networks
- Monte-Carlo dropout uncertainty
- Ensemble uncertainty
- Information-gain exploration



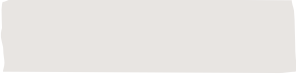
Track 5 — Non-Stationarity

- Sliding-window NeuralBandit
- Discounted NeuralBandit
- Neural change-point detection
- LSTM for changing environments
- Transformer for changing environments
- Meta-learning adaptation
- Continual learning



What is concept drift?

- The relationship between context, action and reward changes over time
- Example: user preferences change
- Example: an advertisement becomes less effective
- Example: a recommendation trend changes
- A good bandit must recover quickly after the environment changes



Track 6 — LLMs as Context Representations

- BERT embeddings
- Sentence-Transformer embeddings
- LLM-generated user representations
- LLM news representations
- LLM product representations
- Text-based contextual bandits
- Compare classical features vs. LLM representations

LLM + NeuralBandit architecture

- Text / user history
- ↓
- LLM or Transformer encoder
- ↓
- Context embedding
- ↓
- NeuralBandit
- ↓
- Action → Reward → Online update

Track 7 — LLMs as Decision Makers

- LLM action scoring
- LLM + UCB
- LLM + Thompson Sampling
- LLM + EXP3
- LLM ensembles for exploration
- LLM vs. neural network policy
- Question: when should an LLM make the decision?

Track 8 — LLM Agents + Bandits

- Tool selection
- Model routing
- Prompt selection
- RAG retriever selection
- Document/chunk selection
- Agent routing
- Multi-agent contextual bandits

Example: bandit for an LLM agent

- User request arrives
- Agent chooses among tools/models
- Tool produces outcome
- Outcome becomes reward
- Bandit learns which tool works best for which context
- Over time: adaptive agent behavior

Track 9 — Efficient LLM Learning

- LoRA NeuralBandit
- QLoRA NeuralBandit
- Quantized LLM bandits
- Parameter-efficient adaptation
- Knowledge distillation
- Large teacher → small student
- Study accuracy, regret, latency and memory

Evaluation: what counts as improvement?

- Cumulative reward
- Cumulative regret
- Instantaneous regret
- Adaptation time after drift
- Sample efficiency
- Inference/training cost
- Memory and latency

Ablation studies

- Do not only report 'our model is better'
- Remove one component at a time
- Example:
 - Transformer + UCB + sliding window
 - → remove Transformer
 - → remove UCB
 - → remove sliding window
- Measure what actually caused the improvement

Baselines

- Original NeuralBandit1
- NeuralBandit2
- NeuralBandit3
- At least one classical contextual-bandit baseline
- Your proposed deep-learning/LLM method
- Use identical environments and evaluation protocols

70 students → one toolkit

- We will use a common API and repository structure
- Each project contributes one module
- Algorithms, models, exploration, non-stationarity, LLMs, applications and evaluation
- The final objective is a reusable NeuralBandit++ toolkit

Suggested repository architecture

- algorithms/ — NeuralBandit, Neural UCB, Neural TS
- models/ — MLP, CNN, LSTM, Transformer, MoE
- exploration/ — UCB, Thompson, EXP3, uncertainty
- nonstationary/ — windows, drift, continual learning
- llm/ — embeddings, routing, RAG, agents
- evaluation/ — regret, reward, statistics, visualization

Your final project workflow

- 1. Read the assigned paper(s)
- 2. Reproduce a baseline
- 3. Establish a benchmark
- 4. Identify a limitation
- 5. Propose a deep-learning/LLM improvement
- 6. Run experiments and ablations
- 7. Analyze results
- 8. Integrate your code into NeuralBandit++

What your final report must answer

- What problem are you solving?
- What did the original method do?
- What is its limitation?
- What deep-learning/LLM idea did you introduce?
- Why should it work?
- How did you evaluate it?
- Did it actually improve reward/regret?
- What are the limitations?

Final deliverable

- Research-quality code
- Experimental results
- Ablation study
- Technical report
- Short presentation/demo
- Contribution integrated into the common toolkit

The ambition

- We are not building 70 isolated homework assignments.
- We are building one modern contextual-bandit platform.
- Your project is one research contribution to that platform.
- The best projects could become workshop/conference-quality research.
- Welcome to NeuralBandit++!